



MHM Aslam

A VPC peering connection pcx-0815d1b46f2aa435b / VPC 1 <> VPC 2 has been requested.
✕

pcx-0815d1b46f2aa435b / VPC 1 <> VPC 2

Actions
✕

🕒
Pending acceptance

You can accept or reject this peering connection request using the 'Actions' menu. You have until Wednesday, May 20, 2026 at 19:21:03 GMT+5:30 to accept or reject the request, otherwise it expires.

Details Info

Requester owner ID
🔗 799227258538

Peering connection ID
🔗 pcx-0815d1b46f2aa435b

Status
🕒 Pending Acceptance by 799227258538

Expiration time
 Wednesday, May 20, 2026 at 19:21:03 GMT+5:30

Accepter owner ID
🔗 799227258538

Requester VPC
🔗 vpc-0408ff3f3cb40c62a / NextWork-1-vpc

Requester CIDRs
🔗 10.1.0.0/16

Requester Region
 N. Virginia (us-east-1)

VPC Peering connection ARN
🔗 arn:aws:ec2:us-east-1:799227258538:vpc-peering-connection/pcx-0815d1b46f2aa435b

Accepter VPC
🔗 vpc-0cbb49d28a9b8427 / NextWork-2-vpc

Accepter CIDRs
 -

Accepter Region
 N. Virginia (us-east-1)

DNS
Route tables
Tags

A circular profile picture of a man with short dark hair, wearing a blue and white sports jersey, standing in front of a green background with some text and graphics.

Introducing Today's Project!

What is Amazon VPC?

Amazon VPC (Virtual Private Cloud) is a service that lets you create a private, isolated section of the AWS cloud where you can launch resources like EC2 instances. It is useful because you have complete control over your network, including choosing IP addresses, creating subnets, and setting up route tables and security groups. This makes your cloud infrastructure secure and customizable, just like a traditional data center but without managing physical hardware.

How I used Amazon VPC in this project

In today's project, I used Amazon VPC to create two separate virtual networks (NextWork-1 and NextWork-2) each with its own CIDR block and one public subnet. I then set up a VPC peering connection between them and updated the route tables so traffic could flow between the VPCs. Finally, I launched EC2 instances in each VPC and tested the peering connection by pinging from one instance to the other using private IP addresses.

One thing I didn't expect in this project was...

One thing I didn't expect in this project was that even after creating the VPC peering connection and updating the route tables, the ping test still failed because the security group on the second instance was blocking ICMP traffic. I had to manually add an inbound rule to allow ping requests from the first VPC's IP range.



MHM Aslam

NextWork Student

nextwork.org

This project took me...

This project took me about 45 minutes to complete, including reading the instructions, creating the VPCs, setting up the peering connection, updating route tables, launching instances, fixing the Elastic IP and security group issues, and finally testing the ping.



In the first part of my project...

Step 1 - Set up my VPC

In this step, I will use the VPC wizard and visual resource map to quickly create two separate VPCs from scratch. I will do this because using the wizard saves time and helps me avoid manual mistakes, so I can focus on learning how to peer the VPCs later.

Step 2 - Create a Peering Connection

In this step, I will create a VPC peering connection to link NextWork-1 and NextWork-2 together. I am doing this because peering allows the two VPCs to communicate directly using private IP addresses, as if they were on the same network, without going through the public internet.

Step 3 - Update Route Tables

In this step, I will update the route tables for both VPCs so that traffic knows how to use the new peering connection. I will add a route in VPC 1's route table that sends traffic destined for VPC 2's CIDR (10.2.0.0/16) to the peering connection, and a similar route in VPC 2's route table for VPC 1's CIDR (10.1.0.0/16). I am doing this because even after the peering connection is created, each VPC's route table still needs an explicit entry to direct traffic across the peer link; otherwise, the traffic will never reach the other VPC.

Step 4 - Launch EC2 Instances

In this step, I will launch one EC2 instance in VPC 1 (NextWork-1) and another EC2 instance in VPC 2 (NextWork-2). I am doing this because I need running servers inside both VPCs to test whether the VPC peering connection actually works.



MHM Aslam

NextWork Student

nextwork.org

By trying to ping or connect from one instance to the other using private IP addresses, I can verify that traffic flows correctly across the peering link.



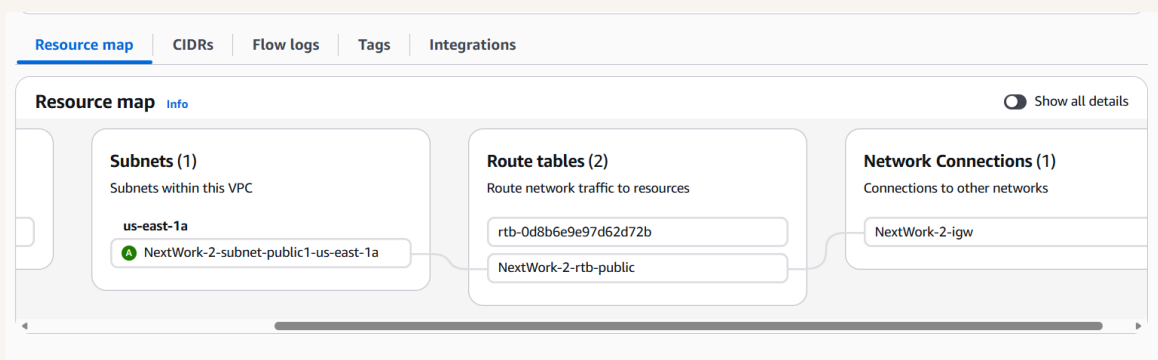
Multi-VPC Architecture

I started my project by launching two VPCs. The first VPC is named NextWork-1 with CIDR 10.1.0.0/16, and the second VPC is NextWork-2 with CIDR 10.2.0.0/16. Each VPC has exactly 1 public subnet and 0 private subnets, so I created a total of 2 subnets.

The CIDR blocks for VPCs 1 and 2 are unique to avoid overlapping IP addresses. They have to be unique because if both VPCs used the same IP range, the VPC peering connection would not know which VPC to send traffic to. Overlapping ranges cause routing conflicts, so keeping them separate ensures clear communication between the two VPCs.

I also launched 2 EC2 instances

I didn't set up key pairs for these EC2 instances as AWS EC2 Instance Connect handles authentication for us automatically. In previous projects I already learned how to create and use key pairs for SSH, but now I can skip that step because Instance Connect lets me securely access the instances without manually managing keys. This saves time while still allowing me to test the VPC peering connection.





VPC Peering

A VPC peering connection is a direct connection between two VPCs that lets them route traffic to each other using private IP addresses. This means data stays within the AWS network and does not go over the public internet.

VPCs would use peering connections to communicate privately and securely using internal IP addresses instead of sending traffic over the public internet. This keeps data transfers faster, cheaper, and more secure.

The difference between a Requester and an Acceptor in a peering connection is that the Requester is the VPC that starts the connection and sends the invitation, while the Acceptor is the VPC that receives the invitation and must accept it for the peering to be established.

Select another VPC to peer with

Account

☒ My account

☐ Another account

Region

☒ This Region (us-east-1)

☐ Another Region

VPC ID (Acceptor)

vpc-0cbab49d28a9b8427 (NextWork-2-vpc) ▼

VPC CIDRs for vpc-0cbab49d28a9b8427 (NextWork-2-vpc)

CIDR	Status	Status reason
10.2.0.0/16	✔ Associated	-



Updating route tables

After accepting a peering connection, my VPCs' route tables need to be updated because the peering link exists but the VPCs do not automatically know how to send traffic to each other. Adding a route tells each VPC which traffic should go through the peering connection. Without these routes, traffic from VPC 1 destined for VPC 2 would have nowhere to go and would be dropped.

My VPCs' new routes have a destination of the other VPC's CIDR block. The routes' target was the VPC peering connection. Specifically, for VPC 1 the destination is 10.2.0.0/16 with target peering connection; for VPC 2 the destination is 10.1.0.0/16 with the same peering connection as target.

Routes	Subnet associations	Edge associations	Route propagation	Tags
Routes (3)				
Filter routes				
Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-051de5ef1f3898484	Active	No	Create Route
10.1.0.0/16	pcx-0815d1b46f2aa435b	Active	No	Create Route
10.2.0.0/16	local	Active	No	Create Route Table



In the second part of my project...

Step 5 - Use EC2 Instance Connect

In this step, I will use EC2 Instance Connect to log into the first EC2 instance (in VPC 1) and fix any connection error that appears. I am doing this because I need to get inside the instance so I can then try to ping or reach the second instance (in VPC 2) to test if the VPC peering connection is working properly.

Step 6 - Connect to EC2 Instance 1

In this step, I will use EC2 Instance Connect again to log into Instance 1 and troubleshoot another error, because even with a public IP, the security group might still block EC2 Instance Connect traffic on port 22, so I need to adjust the inbound rules to allow SSH from the EC2 Instance Connect range.

Step 7 - Test VPC Peering

In this step, I will use Instance 1 to send test messages (like ping or SSH) to Instance 2's private IP address. I will also solve any connection errors that come up until Instance 2 can successfully respond. I am doing this because the only way to confirm that the VPC peering connection and route tables are correctly set up is to actually send traffic between the two VPCs and see if it flows properly.



Troubleshooting Instance Connect

I used EC2 Instance Connect because it lets me securely connect to my EC2 instance directly from the AWS Management Console without needing to manage a key pair or set up an SSH client. It is quick and simple for testing.

I was stopped from using EC2 Instance Connect as the instance did not have a public IPv4 address. After launching the EC2 instance in VPC 1, it had only a private IP, so I could not connect to it from the internet. I fixed this by allocating an Elastic IP and associating it with the instance, which gave it a static public address and allowed EC2 Instance Connect to work.

EC2 Instance Connect

SSM Session Manager

SSH client

EC2 serial console

No public IPv4 or IPv6 address assigned

With no public IPv4 or IPv6 address, you can't use EC2 Instance Connect. Alternatively, you can try connecting using [EC2 Instance Connect Endpoint](#).

Instance ID

i-01d18aa03732bd983 (Instance - NextWork VPC 1)

Connection type

Connect using a Public IP

Connect using a public IPv4 or IPv6 address

Connect using a Private IP

Connect using a private IP address and a VPC endpoint

Public IPv4 address

IPv6 address

Username

Enter the username defined in the AMI used to launch the instance. If you didn't define a custom username, use the default username, ec2-user.

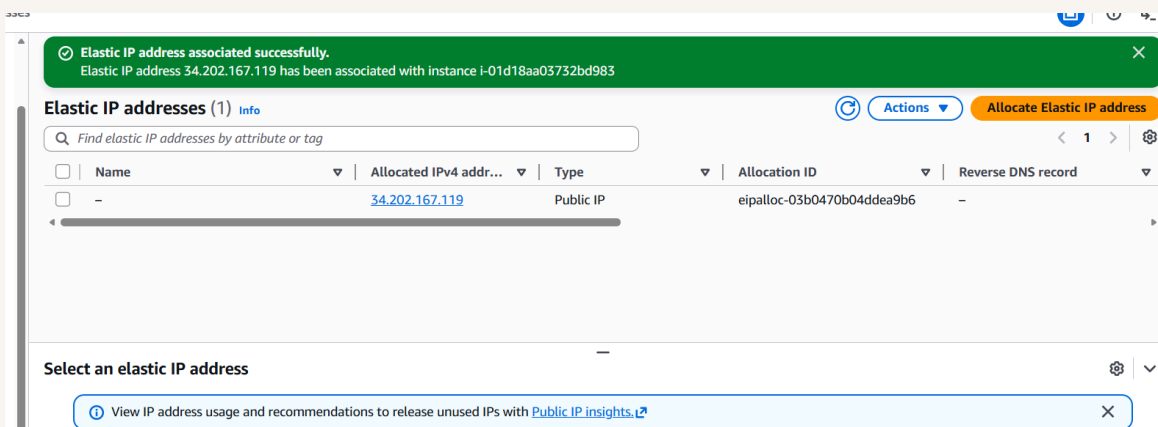
ec2-user



Elastic IP addresses

Elastic IP addresses are static public IPv4 addresses that you can allocate to your AWS account and assign to an EC2 instance. Unlike the default dynamic public IP that changes when an instance stops or restarts, an Elastic IP stays the same, making it useful for consistent internet access and DNS records.

Associating an Elastic IP address resolved the error because the EC2 instance originally had no public IPv4 address, so EC2 Instance Connect could not reach it from the internet. The Elastic IP gave the instance a permanent public address, allowing the connection to be established.





Troubleshooting ping issues

To test VPC peering, I ran the command `ping <private-IP-of-Instance-2>`. For example, `ping 10.2.0.xxx`. This sends ICMP echo requests from Instance 1 to Instance 2 across the peering connection.

A successful ping test would validate my VPC peering connection because ping sends a network request from one instance to another using private IP addresses. If Instance 2 replies, it proves that traffic from Instance 1 can cross the peering link, go through the correct route tables, and reach Instance 2 without being blocked by security groups or network ACLs. This confirms that the two VPCs can communicate privately and directly.

I had to update my second EC2 instance's security group because ping requests were being blocked. I added a new rule that allows ICMP (type 8, code 0) inbound from VPC 1's CIDR (10.1.0.0/16) so that Instance 1 could ping Instance 2 successfully.

```
ec2-user@ip-10-1-11-32 ~]$ ping 10.2.3.243
PING 10.2.3.243 (10.2.3.243) 56(84) bytes of data:
64 bytes from 10.2.3.243: icmp_seq=1 ttl=127 time=0.501 ms
64 bytes from 10.2.3.243: icmp_seq=2 ttl=127 time=0.579 ms
64 bytes from 10.2.3.243: icmp_seq=3 ttl=127 time=0.522 ms
64 bytes from 10.2.3.243: icmp_seq=4 ttl=127 time=0.642 ms
64 bytes from 10.2.3.243: icmp_seq=5 ttl=127 time=0.655 ms
64 bytes from 10.2.3.243: icmp_seq=6 ttl=127 time=0.616 ms
64 bytes from 10.2.3.243: icmp_seq=7 ttl=127 time=0.497 ms
64 bytes from 10.2.3.243: icmp_seq=8 ttl=127 time=0.588 ms
64 bytes from 10.2.3.243: icmp_seq=9 ttl=127 time=0.567 ms
64 bytes from 10.2.3.243: icmp_seq=10 ttl=127 time=0.586 ms
^C
-- 10.2.3.243 ping statistics --
10 packets transmitted, 10 received, 0% packet loss, time 9391ms
rtt min/avg/max/mdev = 0.497/0.575/0.655/0.052 ms
ec2-user@ip-10-1-11-32 ~]$
```

i-01d18aa03732bd983 (Instance - NextWork VPC 1)

PublicIPs: 34.202.167.119 PrivateIPs: 10.1.11.32



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

